

Lesson 1 – PDF Loader (Interview Notes)

1. Why use a class instead of a function?

I used a class because the application is expected to support multiple document types in the future, such as PDF, Markdown, HTML, JSON, or Word documents. By creating separate loader classes with a common `load()` interface, the rest of the pipeline remains independent of the document type. This makes the design more extensible, reusable, and easier to maintain.

2. Why return instead of print?

`print()` is useful only for displaying information during execution or debugging. Returning the extracted text allows other components in the pipeline, such as the chunker, embedding generator, or retriever, to consume and process the data. Returning values makes the function reusable and composable.

3. Why should the loader be stateless?

A stateless loader does not retain information from previous executions. Each call to `load()` starts with a clean state, making the component predictable, reusable, and free from bugs caused by leftover data. Stateless components are also easier to test and maintain.

4. Why does `load()` process one document instead of an entire folder?

Following the **Single Responsibility Principle (SRP)**, each function should perform one well-defined task. The responsibility of `load()` is to extract text from a single document. If multiple documents need to be processed, a separate method such as `load_folder()` can iterate through the directory and internally call `load()` for each file. This keeps the code modular and avoids duplicating logic.

5. What does the `with` statement do?

The `with` statement is a context manager that automatically manages resources. It acquires the resource when entering the block and guarantees that it is released when leaving the block, even if an exception occurs. In this case, it ensures that the PDF file is properly closed after reading, preventing resource leaks.

Key Takeaways

- Design reusable and extensible classes rather than writing one-off functions.
- Return data instead of printing it so that it can be used by other components.
- Prefer stateless components whenever possible.
- Follow the Single Responsibility Principle to keep code modular and maintainable.
- Use context managers (`with`) for safe and automatic resource management.